

WEEK-11.1

2303A51804

Batch-28

Prompt:

```
#generate a python code to underdtand  
stack overflow and underflow operations
```

Code:

```
import sys

def infinite_recursion():

    print("Calling myself again...")
    infinite_recursion()

print("Demonstrating Stack Overflow:")

try:
    infinite_recursion()
except RecursionError:
    print("\nSuccessfully caught RecursionError: Stack Overflow occurred!")
    print("The call stack has exceeded its maximum depth.")
```

Output:

Output:

```
...
Demonstrating Stack Underflow:
Initial stack: []
Caught IndexError: Stack Underflow occurred! (pop from empty list)
Attempted to pop from an empty list/stack.
Stack after appending elements: [10, 20]
Popped element: 20
Stack after successful pop: [10]
```

Task Description #1 – Stack Implementation

Use AI to generate a Stack class with push, pop, peek, and is_empty

Prompt:

```
#generate a Stack class with push, pop, peek, and is_empty
```

Code:

```
class Stack:
    def __init__(self):
        self.items = []

    def is_empty(self):
        """Returns True if the stack is empty, False otherwise."""
        return not self.items

    def push(self, item):
        """Adds an item to the top of the stack."""
        self.items.append(item)
        print(f"Pushed: {item}. Stack: {self.items}")

    def pop(self):
        """Removes and returns the item from the top of the stack.
        Raises IndexError if the stack is empty."""
        if self.is_empty():
            raise IndexError("pop from empty stack")
        popped_item = self.items.pop()
        print(f"Popped: {popped_item}. Stack: {self.items}")
        return popped_item

    def peek(self):
        """Returns the item at the top of the stack without removing it.
        Raises IndexError if the stack is empty."""
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.items[-1]
```

```
stack.push(10)
stack.push(20)
stack.push(30)

print(f"Is stack empty after pushes? {stack.is_empty()}")
print(f"Stack size: {stack.size()}")
print(f"Top element (peek): {stack.peek()}")

stack.pop()
print(f"Top element after pop (peek): {stack.peek()}")

stack.pop()
stack.pop()

print(f"Is stack empty after all pops? {stack.is_empty()}")
try:
    stack.pop()
except IndexError as e:
    print(f"Caught error on empty pop: {e}")
try:
    stack.peek()
except IndexError as e:
    print(f"Caught error on empty peek: {e}")
```

Output:

```
...
--- Demonstrating Stack Class ---
Is stack empty initially? True
Pushed: 10. Stack: [10]
Pushed: 20. Stack: [10, 20]
Pushed: 30. Stack: [10, 20, 30]
Is stack empty after pushes? False
Stack size: 3
Top element (peek): 30
Popped: 30. Stack: [10, 20]
Top element after pop (peek): 20
Popped: 20. Stack: [10]
Popped: 10. Stack: []
Is stack empty after all pops? True
Caught error on empty pop: pop from empty stack
Caught error on empty peek: peek from empty stack
```

Explanation:

- **Initialization:** A Stack object was created, and `is_empty()` confirmed it was initially empty.
- **Push Operations:** Elements 10, 20, and 30 were pushed onto the stack, showing the stack growing and confirming it was no longer empty.
- **Peek and Size:** `peek()` correctly returned 30 (the top element) without removing it, and `size()` showed there were 3 elements.
- **Pop Operations:** Elements were popped from the stack one by one. Each `pop()` operation printed the removed element and the stack's new state. `peek()` was used again to show the new top element after the first pop.
- **Stack Underflow Handling:** Finally, after all elements were popped, attempts to `pop()` and `peek()` from the now-empty stack successfully caught an `IndexError`, demonstrating how the class prevents and handles stack underflow conditions.

Task - 2: Use AI to implement a Queue using Python lists.

Prompt:

```
#Use AI to implement a Queue using Python lists.
```

Code:

```
) class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        """Checks if the queue is empty."""
        return not self.items

    def enqueue(self, item):
        """Adds an item to the rear of the queue."""
        self.items.append(item)
        print(f"Enqueued: {item}. Queue: {self.items}")

    def dequeue(self):
        """Removes and returns the item from the front of the queue.
        Raises IndexError if the queue is empty."""
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        dequeued_item = self.items.pop(0) # Pop from the beginning for FIFO
        print(f"Dequeued: {dequeued_item}. Queue: {self.items}")
        return dequeued_item

    def peek(self):
        """Returns the item at the front of the queue without removing it.
        Raises IndexError if the queue is empty."""
        if self.is_empty():
            raise IndexError("peek from empty queue")
        return self.items[0]

queue.enqueue('A')
queue.enqueue('B')
queue.enqueue('C')

print(f"Is queue empty after enqueues? {queue.is_empty()}")
print(f"Queue size: {queue.size()}")
print(f"Front element (peek): {queue.peek()}")

queue.dequeue()
print(f"Front element after dequeue (peek): {queue.peek()}")

queue.dequeue()
queue.dequeue()

print(f"Is queue empty after all dequeues? {queue.is_empty()}")
try:
    queue.dequeue()
except IndexError as e:
    print(f"Caught error on empty dequeue: {e}")
try:
    queue.peek()
except IndexError as e:
    print(f"Caught error on empty peek: {e}")
```

OUTPUT:

```
--- Demonstrating Queue Class ---  
Is queue empty initially? True  
Enqueued: A. Queue: ['A']  
Enqueued: B. Queue: ['A', 'B']  
Enqueued: C. Queue: ['A', 'B', 'C']  
Is queue empty after enqueues? False  
Queue size: 3  
Front element (peek): A  
Dequeued: A. Queue: ['B', 'C']  
Front element after dequeue (peek): B  
Dequeued: B. Queue: ['C']  
Dequeued: C. Queue: []  
Is queue empty after all dequeues? True  
Caught error on empty dequeue: dequeue from empty queue  
Caught error on empty peek: peek from empty queue
```

Task-3: Use AI to generate a Singly Linked List with insert and display

Prompt:

#Use AI to generate a Singly Linked List with insert and display

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        """Inserts a new node with the given data at the end of the list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            print(f"Inserted {data}. List is now: {self.display_list()}")
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
        print(f"Inserted {data}. List is now: {self.display_list()}")

    def display_list(self):
        """Displays the contents of the linked list."""
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
```



```

def display_list(self):
    """Displays the contents of the linked list."""
    elements = []
    current = self.head
    while current:
        elements.append(str(current.data))
        current = current.next
    return " -> ".join(elements) if elements else "Empty List"

print("\n--- Demonstrating Singly Linked List ---")
my_list = LinkedList()
print(f"Initial list: {my_list.display_list()}")

my_list.insert_at_end(10)
my_list.insert_at_end(20)
my_list.insert_at_end(30)

print(f"List after insertions: {my_list.display_list()}")

my_list.insert_at_end(40)
print(f"List after one more insertion: {my_list.display_list()}")

return " -> ".join(elements) if elements else "Empty List"

print("\n--- Demonstrating Singly Linked List ---")
my_list = LinkedList()
print(f"Initial list: {my_list.display_list()}")

my_list.insert_at_end(10)
my_list.insert_at_end(20)
my_list.insert_at_end(30)

print(f"List after insertions: {my_list.display_list()}")

my_list.insert_at_end(40)
print(f"List after one more insertion: {my_list.display_list()}")

```

Output:

```

--- Demonstrating Singly Linked List ---
Initial list: Empty List
Inserted 10. List is now: 10
Inserted 20. List is now: 10 -> 20
Inserted 30. List is now: 10 -> 20 -> 30
List after insertions: 10 -> 20 -> 30
Inserted 40. List is now: 10 -> 20 -> 30 -> 40
List after one more insertion: 10 -> 20 -> 30 -> 40

```

Explanation:

- **next:** This is a reference (or pointer) to the *next* Node in the sequence. For the last node in the list, next will be None, indicating the end.

- `__init__(self)`: The constructor initializes the linked list with a head attribute set to None. The head is a reference to the very first Node in the list. If head is None, the list is empty.

Task 4: Use AI to create a BST with insert and in-order traversal methods.

Prompt:

Use AI to create a BST with insert and in-order traversal methods.

Code:

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):

        new_node = TreeNode(key)
        if self.root is None:
            self.root = new_node
            print(f"Inserted {key}. Root is now {key}.")
            return

        current = self.root
        parent = None
        while current:
            parent = current
            if key < current.key:
                current = current.left
            elif key > current.key:
                current = current.right
            else:
                print(f"Key {key} already exists in the tree. No duplicates allowed.")
                return
```



```
        if node:
            self._in_order_traverse(node.left, result)
            result.append(node.key)
            self._in_order_traverse(node.right, result)

print("\n--- Demonstrating Binary Search Tree ---")
bst = BinarySearchTree()

print("Inserting elements:")
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)
bst.insert(30)

print("\nIn-order traversal:")
traversed_elements = bst.in_order_traversal()
print(f"Elements in ascending order: {traversed_elements}")
```

```
Inserting elements:
Inserted 50. Root is now 50.
Inserted 30.
Inserted 70.
Inserted 20.
Inserted 40.
Inserted 60.
Inserted 80.
Key 30 already exists in the tree. No duplicates allowed.
```

```
In-order traversal:
Elements in ascending order: [20, 30, 40, 50, 60, 70, 80]
```



```
        if node:
            self._in_order_traverse(node.left, result)
            result.append(node.key)
            self._in_order_traverse(node.right, result)

print("\n--- Demonstrating Binary Search Tree ---")
bst = BinarySearchTree()

print("Inserting elements:")
bst.insert(50)
bst.insert(30)
bst.insert(70)
bst.insert(20)
bst.insert(40)
bst.insert(60)
bst.insert(80)
bst.insert(30)

print("\nIn-order traversal:")
traversed_elements = bst.in_order_traversal()
print(f"Elements in ascending order: {traversed_elements}")
```

Output:

```
--- Demonstrating Binary Search Tree ---
Inserting elements:
Inserted 50. Root is now 50.
Inserted 30.
Inserted 70.
Inserted 20.
Inserted 40.
Inserted 60.
Inserted 80.
Key 30 already exists in the tree. No duplicates allowed.

In-order traversal:
Elements in ascending order: [20, 30, 40, 50, 60, 70, 80]
```

TASK-5: Use AI to implement a hash table with basic insert, search, and delete methods.

Prompt:

#implement a hash table with basic insert, search, and delete methods.💡

Code:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(self.size)]

    def _hash_function(self, key):

        return hash(key) % self.size

    def insert(self, key, value):

        hash_index = self._hash_function(key)
        for i, (k, v) in enumerate(self.table[hash_index]):
            if k == key:
                self.table[hash_index][i] = (key, value)
                print(f"Updated key '{key}' with value '{value}'.")
                return
        self.table[hash_index].append((key, value))
        print(f"Inserted ('{key}', '{value}') at index {hash_index}.")

    def search(self, key):

        hash_index = self._hash_function(key)
        for k, v in self.table[hash_index]:
            if k == key:
                print(f"Found key '{key}' with value '{v}' at index {hash_index}.")
                return v
        print(f"Key '{key}' not found.")
```

```

def display(self):

    print("\n--- Hash Table Contents ---")
    for i, bucket in enumerate(self.table):
        print(f"Bucket {i}: {bucket}")

print("--- Demonstrating Hash Table ---")
hash_table = HashTable(size=5)

hash_table.insert("apple", 10)
hash_table.insert("banana", 20)
hash_table.insert("cherry", 30)
hash_table.insert("date", 40)

hash_table.display()

hash_table.insert("apple", 15) # Update existing key
hash_table.insert("elderberry", 50) # Collision might occur

hash_table.display()

print("\n--- Searching ---")
hash_table.search("banana")
hash_table.search("grape")

print("\n--- Deleting ---")
hash_table.delete("cherry")

```

Output:

Inserted ('apple', '10') at index 0.
Inserted ('banana', '20') at index 3.
Inserted ('cherry', '30') at index 2.
Inserted ('date', '40') at index 1.

--- Hash Table Contents ---

Bucket 0: [('apple', 10)]
Bucket 1: [('date', 40)]
Bucket 2: [('cherry', 30)]
Bucket 3: [('banana', 20)]
Bucket 4: []
Updated key 'apple' with value '15'.
Inserted ('elderberry', '50') at index 3.

--- Hash Table Contents ---

Bucket 0: [('apple', 15)]
Bucket 1: [('date', 40)]
Bucket 2: [('cherry', 30)]
Bucket 3: [('banana', 20), ('elderberry', 50)]
Bucket 4: []

--- Searching ---

Found key 'banana' with value '20' at index 3.
Key 'grape' not found.

--- Deleting ---

Deleted key 'cherry' from index 2.
Key 'fig' not found for deletion.

--- Hash Table Contents ---

Bucket 0: [('apple', 15)]
Bucket 1: [('date', 40)]

...

```
--- Searching ---  
Found key 'banana' with value '20' at index 3.  
Key 'grape' not found.  
  
--- Deleting ---  
Deleted key 'cherry' from index 2.  
Key 'fig' not found for deletion.  
  
--- Hash Table Contents ---  
Bucket 0: [('apple', 15)]  
Bucket 1: [('date', 40)]  
Bucket 2: []  
Bucket 3: [('banana', 20), ('elderberry', 50)]  
Bucket 4: []  
Deleted key 'apple' from index 0.  
  
--- Hash Table Contents ---  
Bucket 0: []  
Bucket 1: [('date', 40)]  
Bucket 2: []  
Bucket 3: [('banana', 20), ('elderberry', 50)]  
Bucket 4: []
```

Explanation:

1. **Initialization:** A HashTable with 5 buckets was created.
2. **Insertions:** Several key-value pairs like ('apple', 10), ('banana', 20), etc., were inserted. You saw how each was placed into a specific bucket based on its hash.
3. **Update:** Inserting 'apple' again with value 15 showed that the existing 'apple' entry was updated, not duplicated.
4. **Collision Example:** 'elderberry' was inserted, and because its hash likely collided with 'banana', both ended up in the same bucket (bucket 3), stored as [('banana', 20), ('elderberry', 50)].
5. **Searching:** You saw a successful search for 'banana' and an unsuccessful search for 'grape'.
6. **Deletion:** 'cherry' was successfully removed, and an attempt to delete 'fig' (which didn't exist) resulted in a 'not found' message. Finally, 'apple' was also deleted.

Task-6: Use AI to implement a graph using an adjacency list.

Prompt:

#implement a graph using an adjacency list.💡

Code:

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        """Adds a vertex to the graph if it doesn't already exist."""
        if vertex not in self.graph:
            self.graph[vertex] = []
            print(f"Added vertex: {vertex}")
        else:
            print(f"Vertex {vertex} already exists.")

    def add_edge(self, vertex1, vertex2):

        if vertex1 not in self.graph:
            self.add_vertex(vertex1)
        if vertex2 not in self.graph:
            self.add_vertex(vertex2)

        if vertex2 not in self.graph[vertex1]:
            self.graph[vertex1].append(vertex2)
            print(f"Added edge {vertex1} -> {vertex2}")
        else:
            print(f"Edge {vertex1} -> {vertex2} already exists.")

        if vertex1 not in self.graph[vertex2]:
            self.graph[vertex2].append(vertex1)
            print(f"Added edge {vertex2} -> {vertex1}")
```

```
print("--- Demonstrating Graph Implementation ---")
g = Graph()

g.add_vertex('A')
g.add_vertex('B')
g.add_vertex('C')
g.add_vertex('D')

g.add_edge('A', 'B')
g.add_edge('B', 'C')
g.add_edge('A', 'D')
g.add_edge('D', 'C')
g.add_edge('B', 'D')

g.add_vertex('A')
g.add_edge('A', 'B')

g.display_graph()

print("\n--- Demonstrating another graph (disconnected components) ---")
g2 = Graph()
g2.add_vertex(1)
g2.add_vertex(2)
g2.add_vertex(3)
g2.add_edge(1, 2)
g2.add_vertex(4)
g2.display_graph()
```

Output:

```
Added vertex: A
Added vertex: B
Added vertex: C
Added vertex: D
Added edge A -> B
Added edge B -> A
Added edge B -> C
Added edge C -> B
Added edge A -> D
Added edge D -> A
Added edge D -> C
Added edge C -> D
Added edge B -> D
Added edge D -> B
Vertex A already exists.
Edge A -> B already exists.

--- Graph Adjacency List ---
A: B, D
B: A, C, D
C: B, D
D: A, C, B

--- Demonstrating another graph (disconnected components) ---
Added vertex: 1
Added vertex: 2
Added vertex: 3
Added edge 1 -> 2
Added edge 2 -> 1
Added vertex: 4

Added vertex: 4

--- Graph Adjacency List ---
1: 2
2: 1
3:
4:
```

Explanation:

- **Vertex and Edge Addition:** Successfully added vertices ('A', 'B', 'C', 'D', and 1, 2, 3, 4) and undirected edges between them.
- **Existing Elements:** The system correctly identified and reported attempts to add duplicate vertices ('A') or edges ('A' -> 'B'), preventing redundancy.

- **Adjacency List Representation:** The display `_graph` method accurately presented the graph's structure, showing each vertex and its connected neighbours.
- **Disconnected Components:** Demonstrated the ability to represent disconnected parts of a graph, such as vertices 3 and 4 in the second example, which had no outgoing or incoming edges.

Task-7: Implement a priority queue using Python's `heapq` module

Prompt:

implement a priority queue using Python's `heapq` module.

Code:

```
import heapq
class PriorityQueue:
    def __init__(self):
        self._queue = []
        self._index = 0

    def push(self, item, priority):
        heapq.heappush(self._queue, (priority, self._index, item))
        print(f"Pushed: '{item}' with priority {priority}. Queue state: {[p, i, it] for p, i, it in self._queue}")
        self._index += 1

    def pop(self):
        if self.is_empty():
            raise IndexError("pop from empty priority queue")
        priority, _, item = heapq.heappop(self._queue)
        print(f"Popped: '{item}' with priority {priority}. Queue state: {[p, i, it] for p, i, it in self._queue}")
        return item

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty priority queue")
        return self._queue[0][2]

    def is_empty(self):
        return not self._queue

    def size(self):
        return len(self._queue)
```

```

print(f"Is priority queue empty initially? {pq.is_empty()}")
pq.push('Task A', 3)
pq.push('Task B', 1)
pq.push('Task C', 2)
pq.push('Task D', 1)
pq.push('Task E', 5)
print(f"Is priority queue empty after pushes? {pq.is_empty()}")
print(f"Priority queue size: {pq.size()}")
print(f"Highest priority item (peek): {pq.peek()}")
print("\n--- Popping items by priority ---")
print(f"Popped: {pq.pop()}")
print(f"Highest priority item (peek): {pq.peek()}")
print(f"Popped: {pq.pop()}")
print(f"Highest priority item (peek): {pq.peek()}")
print(f"Popped: {pq.pop()}")
print("\n--- Remaining items ---")
print(f"Priority queue size: {pq.size()}")
print(f"Highest priority item (peek): {pq.peek()}")

print(f"Popped: {pq.pop()}")
print(f"Popped: {pq.pop()}")

print(f"Is priority queue empty after all pops? {pq.is_empty()}")
try:
    pq.pop()
except IndexError as e:
    print(f"Caught error on empty pop: {e}")
try:
    pq.peek()

```

Output:

```

--- Demonstrating Priority Queue ---
Is priority queue empty initially? True
Pushed: 'Task A' with priority 3. Queue state: [(3, 0, 'Task A')]
Pushed: 'Task B' with priority 1. Queue state: [(1, 1, 'Task B'), (3, 0, 'Task A')]
Pushed: 'Task C' with priority 2. Queue state: [(1, 1, 'Task B'), (3, 0, 'Task A'), (2, 2, 'Task C')]
Pushed: 'Task D' with priority 1. Queue state: [(1, 1, 'Task B'), (1, 3, 'Task D'), (2, 2, 'Task C'), (3, 0, 'Task A')]
Pushed: 'Task E' with priority 5. Queue state: [(1, 1, 'Task B'), (1, 3, 'Task D'), (2, 2, 'Task C'), (3, 0, 'Task A'), (5, 4, 'Task E')]
Is priority queue empty after pushes? False
Priority queue size: 5
Highest priority item (peek): Task B

--- Popping items by priority ---
Popped: 'Task B' with priority 1. Queue state: [(1, 3, 'Task D'), (3, 0, 'Task A'), (2, 2, 'Task C'), (5, 4, 'Task E')]
Popped: Task B
Highest priority item (peek): Task D
Popped: 'Task D' with priority 1. Queue state: [(2, 2, 'Task C'), (3, 0, 'Task A'), (5, 4, 'Task E')]
Popped: Task D
Highest priority item (peek): Task C
Popped: 'Task C' with priority 2. Queue state: [(3, 0, 'Task A'), (5, 4, 'Task E')]
Popped: Task C

--- Remaining items ---
Priority queue size: 2
Highest priority item (peek): Task A
Popped: 'Task A' with priority 3. Queue state: [(5, 4, 'Task E')]
Popped: Task A
Popped: 'Task E' with priority 5. Queue state: []
Popped: Task E
Is priority queue empty after all pops? True
Caught error on empty pop: pop from empty priority queue
Caught error on empty peek: peek from empty priority queue

```

Explanation:

- **smallest Priority First:** The `heapq` module implements a min-heap, meaning items with the *lowest* numerical priority value are considered 'highest priority' and are popped first.
- **Stable Sorting:** If multiple items have the *same* priority, their original insertion order is preserved. For instance, 'Task B' and 'Task D' both had priority 1, but 'Task B' was pushed first, so it was popped first.
- **Empty Queue Handling:** The implementation correctly raises `IndexError` if `pop()` or `peek()` are called on an empty queue.

Task-8:– Deque

PROMPT: Implement a double-ended queue using `collections.deque`.

Code:

```
238
239 #implement a double-ended queue using collections.deque
240 from collections import deque
241 class Deque:
242     def __init__(self):
243         self.deque = deque()
244
245     def append(self, item):
246         self.deque.append(item)
247
248     def appendleft(self, item):
249         self.deque.appendleft(item)
250
251     def pop(self):
252         if not self.deque:
253             raise IndexError("Deque is empty.")
254         return self.deque.pop()
255
256     def popleft(self):
257         if not self.deque:
258             raise IndexError("Deque is empty.")
259         return self.deque.popleft()
260
261     def peek(self):
262         if not self.deque:
263             raise IndexError("Deque is empty.")
264         return self.deque[0]
265
266     def peek_right(self):
267         if not self.deque:
268             raise IndexError("Deque is empty.")
269         return self.deque[-1]
270
271     def is_empty(self):
272         return len(self.deque) == 0
273
274 # Example usage:
275 deque_instance = Deque()
276 deque_instance.append(1)
277 deque_instance.append(2)
278 deque_instance.appendleft(0)
279 print(deque_instance.peek()) # Output: 0
280 print(deque_instance.peek_right()) # Output: 2
281 print(deque_instance.pop()) # Output: 2
282 print(deque_instance.popleft()) # Output: 0
283 print(deque_instance.is_empty()) # Output: False
284
```

Explanation:

- The `append` and `appendleft` methods add items to the right and left ends of the deque, respectively, while the `pop` and `popleft` methods remove items from the right and left ends.
- The `peek` and `peek_right` methods allow you to see the front and back items of the deque without removing them, and the `is_empty` method checks if the deque is empty.
- The `Deque` class uses the `collections.deque` data structure, which allows for efficient appending and popping from both ends of the queue.

Task-9:Time Application Challenge – Choose the Right Data Structure

PROMPT:

generate a code to develop a campus resource management system that handles:

Student Attendance Tracking – Daily log of students entering/exiting the campus.

Event Registration System – Manage participants in events with quick search and removal.

Library Book Borrowing – Keep track of available books and their due dates.

Bus Scheduling System – Maintain bus routes and stop connections.

Cafeteria Order Queue – Serve students in the order they arrive.

For each feature, select the most appropriate data structure from the list below

Stack ,queue, linked list, binary search tree, hash table, graph, deque

Code:

```
#generate a code to develop a campus resource management system that handles:  
#Student Attendance Tracking - Daily log of students entering/exiting the campus.  
#Event Registration System - Manage participants in events with quick search and removal.  
#Library Book Borrowing - Keep track of available books and their due dates.  
#Bus Scheduling System - Maintain bus routes and stop connections.  
#Cafeteria Order Queue - Serve students in the order they arrive.  
#For each feature, select the most appropriate data structure from the list below  
#Stack ,queue, linked list, binary search tree, hash table, graph, deque
```

```
class StudentAttendance:  
    def __init__(self):  
        self.attendance_log = [] # Using a list to store attendance records  
  
    def log_entry(self, student_id):  
        self.attendance_log.append((student_id, "entry"))  
  
    def log_exit(self, student_id):  
        self.attendance_log.append((student_id, "exit"))  
  
    def display_attendance(self):  
        print("Student ID | Status")  
        print("-----")  
        for record in self.attendance_log:  
            print(f"{record[0]:<10} | {record[1]}")  
  
# Example usage:  
attendance_system = StudentAttendance()  
attendance_system.log_entry("S001")  
attendance_system.log_entry("S002")  
attendance_system.log_exit("S001")  
attendance_system.display_attendance()
```

```
#code for event registration system  
class EventRegistration:  
    def __init__(self):  
        self.participants = set() # Using a set for quick search and removal  
  
    def register_participant(self, participant_id):  
        self.participants.add(participant_id)  
  
    def remove_participant(self, participant_id):  
        self.participants.discard(participant_id) # discard does not raise an error if the item is not found  
  
    def display_participants(self):  
        print("Registered Participants:")  
        for participant in self.participants:  
            print(participant)  
  
# Example usage:  
event_registration = EventRegistration()  
event_registration.register_participant("P001")  
event_registration.register_participant("P002")  
event_registration.remove_participant("P001")  
event_registration.display_participants()  
  
#code for library book borrowing  
class Library:  
    def __init__(self):  
        self.books = {} # Using a dictionary to keep track of books and their due dates  
  
    def add_book(self, book_id, due_date):  
        self.books[book_id] = due_date  
  
    def borrow_book(self, book_id):  
        if book_id in self.books:  
            del self.books[book_id] # Remove the book from available books  
        else:  
            print(f"Book {book_id} is not available.")  
  
    def display_books(self):  
        print("Available Books:")  
        for book_id, due_date in self.books.items():  
            print(f"Book ID: {book_id}, Due Date: {due_date}")  
  
# Example usage:  
library = Library()  
library.add_book("B001", "2024-07-01")  
library.add_book("B002", "2024-07-15")  
library.borrow_book("B001")  
library.display_books()
```



```

#code for bus scheduling system
class BusSchedule:
    def __init__(self):
        self.routes = {} # Using a dictionary to maintain bus routes and stop connections

    def add_route(self, route_id, stops):
        self.routes[route_id] = stops

    def display_routes(self):
        print("Bus Routes:")
        for route_id, stops in self.routes.items():
            print(f"Route ID: {route_id}, Stops: {' '.join(stops)}")

# Example usage:
bus_schedule = BusSchedule()
bus_schedule.add_route("R001", ["Stop A", "Stop B", "Stop C"])
bus_schedule.add_route("R002", ["Stop D", "Stop E"])
bus_schedule.display_routes()

#code for cafeteria order queue
from collections import deque
class CafeteriaOrderQueue:
    def __init__(self):
        self.order_queue = deque() # Using a deque to serve students in the order they arrive

    def place_order(self, student_id, order):
        self.order_queue.append((student_id, order))

    def serve_order(self):
        if self.order_queue:
            return self.order_queue.popleft() # Serve the next order in line
        else:
            print("No orders to serve.")

    def display_orders(self):
        print("Current Orders in Queue:")
        for student_id, order in self.order_queue:
            print(f"Student ID: {student_id}, Order: {order}")

# Example usage:
cafeteria_queue = CafeteriaOrderQueue()
cafeteria_queue.place_order("S001", "Sandwich")
cafeteria_queue.place_order("S002", "Salad")
cafeteria_queue.display_orders()
print(cafeteria_queue.serve_order()) # Output: ('S001', 'Sandwich')
cafeteria_queue.display_orders()

```

Output:

```

PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> ^C
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> &

C:\Users\Pavani\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Pavani\OneDrive\Documents\AI Assisted coding\lab11-1.py"

Student ID | Status
-----
S001      | entry
S002      | entry
S003      | exit
Registered Participants:
P002
Available Books:
Book ID: B002, Due Date: 2024-07-15
Bus Routes:
Route ID: R001, Stops: Stop A, Stop B, Stop C
Route ID: R002, Stops: Stop D, Stop E
Current Orders in Queue:
Student ID: S001, Order: Sandwich
Student ID: S002, Order: Salad
('S001', 'Sandwich')
Current Orders in Queue:
Student ID: S002, Order: Salad
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding>

```

Explanation:

- Each class executes a specific function of the campus resource management system
- The StudentAttendance class manages the attendance log of students entering and exiting the campus using a list to store records.
- The EventRegistration class uses a set to manage event participants, allowing for quick registration, removal, and display of participants.
- The Library class maintains a dictionary to track available books and their due dates, while
- The BusSchedule class uses a dictionary to manage bus routes and their stops, and the CafeteriaOrderQueue class utilizes a deque to serve students in the order they arrive

Task 10: Smart E-Commerce Platform – Data Structure Challenge

PROMPT:

write a python code to build a smart online shopping system that includes:

Shopping Cart Management – Add and remove products dynamically

Order Processing System – Orders processed in the order they are placed

Top-Selling Products Tracker – Products ranked by sales count.

Product Search Engine – Fast lookup of products using product ID.

Delivery Route Planning – Connect warehouses and delivery locations

For each feature, select the most appropriate data structure from the list below

stack, queue, priority queue, linkedlist, BST, Graph, Hash Table, Deque

Code:

```
#write a python code to build a smart online shopping system that includes:  
#Shopping Cart Management - Add and remove products dynamically  
#Order Processing System - Orders processed in the order they are placed  
#Top-Selling Products Tracker - Products ranked by sales count.  
#Product Search Engine - Fast lookup of products using product ID.  
#Delivery Route Planning - Connect warehouses and delivery locations  
#For each feature, select the most appropriate data structure from the list below  
#stack, queue,priority queue, linkedlist,BST,Graph,Hash Table, Deque  
from collections import deque
```

```
class ShoppingCart:  
    def __init__(self):  
        self.cart = [] # Using a list to manage shopping cart items  
  
    def add_product(self, product_id):  
        self.cart.append(product_id)  
  
    def remove_product(self, product_id):  
        if product_id in self.cart:  
            self.cart.remove(product_id)  
        else:  
            print(f"Product {product_id} not found in cart.")  
  
    def display_cart(self):  
        print("Shopping Cart:")  
        for product in self.cart:  
            print(f"- Product ID: {product}")  
  
# Example usage:  
cart = ShoppingCart()  
cart.add_product("P001")  
cart.add_product("P002")  
cart.display_cart()  
cart.remove_product("P001")  
cart.display_cart()
```

```

class OrderProcessing:
    def __init__(self):
        self.order_queue = deque() # Using a deque to process orders in the order they are placed

    def place_order(self, order_id):
        self.order_queue.append(order_id)

    def process_order(self):
        if self.order_queue:
            return self.order_queue.popleft() # Process the next order in line
        else:
            print("No orders to process.")

    def display_orders(self):
        print("Current Orders in Queue:")
        for order_id in self.order_queue:
            print(f"- Order ID: {order_id}")

# Example usage:
order_processing = OrderProcessing()
order_processing.place_order("0001")
order_processing.place_order("0002")
order_processing.display_orders()
print(order_processing.process_order()) # Output: 0001
order_processing.display_orders()

class TopSellingProducts:
    def __init__(self):
        self.sales_count = {} # Using a dictionary to track sales count of products

    def record_sale(self, product_id):
        if product_id in self.sales_count:
            self.sales_count[product_id] += 1
        else:
            self.sales_count[product_id] = 1

    def top_selling_products(self, n=5):
        sorted_products = sorted(self.sales_count.items(), key=lambda x: x[1], reverse=True)
        return sorted_products[:n]

# Example usage:
top_selling = TopSellingProducts()
top_selling.record_sale("P001")
top_selling.record_sale("P002")
top_selling.record_sale("P001")
print(top_selling.top_selling_products()) # Output: [('P001', 2

```

```

14 class ProductSearchEngine:
15     def __init__(self):
16         self.products = {} # Using a dictionary for fast lookup of products by ID
17
18     def add_product(self, product_id, product_info):
19         self.products[product_id] = product_info
20
21     def search_product(self, product_id):
22         return self.products.get(product_id, "Product not found.")
23
24 # Example usage:
25 search_engine = ProductSearchEngine()
26 search_engine.add_product("P001", {"name": "Laptop", "price": 999})
27 search_engine.add_product("P002", {"name": "Smartphone", "price": 499})
28 print(search_engine.search_product("P001")) # Output: {'name': 'Laptop', 'price': 999}
29
30 class DeliveryRoutePlanning:
31     def __init__(self):
32         self.graph = {} # Using a graph to connect warehouses and delivery locations
33
34     def add_location(self, location):
35         if location not in self.graph:
36             self.graph[location] = []
37
38     def add_route(self, location1, location2):
39         if location1 in self.graph and location2 in self.graph:
40             self.graph[location1].append(location2)
41             self.graph[location2].append(location1) # For undirected graph
42
43     def display_routes(self):
44         print("Delivery Routes:")
45         for location, routes in self.graph.items():
46             print(f"{location}: {' '.join(routes)}")
47
48 # Example usage:
49 route_planning = DeliveryRoutePlanning()
50 route_planning.add_location("Warehouse A")
51 route_planning.add_location("Warehouse B")
52 route_planning.add_location("Customer X")
53 route_planning.add_route("Warehouse A", "Customer X")
54 route_planning.add_route("Warehouse B", "Customer X")
55 route_planning.display_routes()

```

Output:

```
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> & C:/Users/Pavani/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Pavani/OneDrive/Documents/AI Assisted coding/lab11-1
Shopping Cart:
- Product ID: P001
- Product ID: P002
Shopping Cart:
- Product ID: P002
Current Orders in Queue:
- Order ID: 0001
- Order ID: 0002
0001
Current Orders in Queue:
- Order ID: 0002
[('P001', 2), ('P002', 1)]
{'name': 'Laptop', 'price': 999}
Delivery Routes:
Warehouse A: Customer X
Warehouse B: Customer X
Customer X: Warehouse A, Warehouse B
PS C:\Users\Pavani\OneDrive\Documents\AI Assisted coding> []
```

Explanation:

Each class executes a specific function of the smart online shopping system

The ShoppingCart class manages the shopping cart using a list to add and remove products dynamically,

while the OrderProcessing class uses a deque to process orders in the order they are placed.

The TopSellingProducts class maintains a dictionary to track sales count of products and provides a method to retrieve the top-selling products,

while the ProductSearchEngine class uses a dictionary for fast lookup of products by their ID.

The DeliveryRoutePlanning class implements a graph to connect warehouses and delivery locations, allowing for the management of delivery routes.